

INSTITUTO TECNOLÓGICO DE AERONÁUTICA
CURSO DE ENGENHARIA ELETRÔNICA

RELATÓRIO DE ESTÁGIO

**Instituto de Fomento e Coordenação
Industrial – IFI/DCTA**



Nome do Aluno: Andryus Marzzona Fernandes Martins
São José dos Campos, SP

FOLHA DE APROVAÇÃO

Relatório Final de Estágio Curricular aceito em 15/05/2026 pelos abaixo assinados:

Andryus Marzzona Fernandes Martins

Ten. Cássio Vinícius do Nascimento Santos – Orientador/Supervisor na Empresa

Daniel Basso Ferreira – Orientador/Supervisor no ITA

Marcelo da Silva Pinho – Coordenador do Curso de Engenharia Eletrônica

INFORMAÇÕES GERAIS

Estagiário

Nome do Aluno: Andryus Marzzona Fernandes Martins

Curso: Engenharia Eletrônica

Turma: ELE 2026

Empresa/Departamento

Instituto de Fomento e Coordenação Industrial – IFI/DCTA

Endereço da Empresa: Praça Mal. Eduardo Gomes, 50, Vila das Acácias

Cidade: São José dos Campos Estado: SP CEP: 12.231-970

CNPJ: 00.394.429/0142-41

Orientador/Supervisor da Empresa

Ten. Cássio Vinícius do Nascimento Santos

Cargo: Chefe da Subdivisão de Laboratórios de Ensaios

Tel: 12 98169-8226 e-mail: cassiocvns@fab.mil.br

Orientador/Supervisor do ITA

Daniel Basso Ferreira

Ramal: 6873 e-mail: danielbf@ita.br

Período

02/03/2026 a 24/04/2026

Total de horas: 172 horas

I. INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio curricular realizado no Laboratório de Medições Eletromagnéticas (DCTA/IFI/CMA-LE), pertencente ao Instituto de Fomento e Coordenação Industrial (IFI/DCTA). O laboratório dispõe de uma câmara anecoica utilizada tanto na realização de ensaios de compatibilidade eletromagnética (EMC) segundo a norma militar MIL-STD-461 (Department of Defense, 2015), quanto na caracterização de antenas em campo distante (Oliveira and Santos, 2020).

Antes do estágio, as medições de caracterização de antenas no laboratório eram conduzidas com medidor de potência, instrumento que fornece apenas a amplitude do sinal recebido. Para determinar a razão axial de uma antena de polarização circular ou elíptica, era necessário empregar o método da antena linear girante (IEEE, 2021): faz-se girar mecanicamente a antena transmissora, de polarização linear, enquanto se registra a amplitude da potência recebida em função do ângulo de rotação; a razão entre o valor máximo e o valor mínimo da envoltória fornece a razão axial naquela frequência. Esse método, embora correto, exige sessões de medição dedicadas, não fornece informação de fase e, por basear-se exclusivamente em amplitude, não permite determinar o sentido da polarização elíptica — se à direita ou à esquerda (IEEE, 2021).

A principal atividade de desenvolvimento realizada ao longo do estágio foi a implementação de um sistema de software para automatizar medições de parâmetros S em câmara anecoica com um analisador de redes vetoriais (VNA — *Vector Network Analyzer*). Ao contrário do medidor de potência, o VNA mede amplitude e fase do sinal transmitido entre as antenas, fornecendo os parâmetros S completos em representação complexa (IEEE, 2021). Com duas varreduras angulares completas da antena receptora — uma para cada polarização linear ortogonal da transmissora — todas as grandezas necessárias à caracterização da antena sob teste ficam disponíveis em uma única sessão de medição, sem necessidade de procedimentos adicionais: o diagrama de irradiação, a polarização e a razão axial em função da frequência.

Paralelamente ao desenvolvimento do sistema, acompanhou-se a equipe do laboratório na condução de ensaios de compatibilidade eletromagnética, entre os quais os ensaios de susceptibilidade conduzida CS114 e CS116 e os ensaios de emissão irradiada e susceptibilidade irradiada RE102 e RS103, todos realizados segundo os requisitos da norma MIL-STD-461 (Department of Defense, 2015).

II. A EMPRESA

II.1. Histórico

A origem do IFI remonta ao esforço de estruturação do setor aeronáutico brasileiro no pós-guerra (Instituto de Fomento e Coordenação Industrial, 2025). Em 29 de janeiro de 1946, o Ministro da Aeronáutica, Major Brigadeiro-do-Ar Armando F. Trompowsky de Almeida, criou, pela Portaria nº 36, a Comissão de Organização do Centro Técnico de Aeronáutica (COCTA). Composta por três engenheiros militares e assessorada pelo Professor Richard H. Smith, do Massachusetts Institute of Technology (MIT), a comissão tinha a missão de receber os terrenos doados pelo Estado de São Paulo, em São José dos Campos, e planejar a instalação do futuro Centro Técnico de Aeronáutica (CTA).

A visão de Smith para o CTA fundamentava-se num trinômio indissociável — ensino, pesquisa e produção — que orientaria a criação progressiva de seus institutos. O segmento ensino foi atendido com a fundação do Instituto Tecnológico de Aeronáutica (ITA), pela Lei 2.165, de 5 de janeiro de 1954. O segmento pesquisa veio a ser coberto pelo Instituto de Pesquisa e Desenvolvimento (IPD), criado em 23 de agosto de 1965. Faltava, ainda, um instituto voltado ao elo entre a pesquisa e o setor produtivo.

Essa lacuna começou a ser preenchida com as reformas administrativas decorrentes do Decreto-Lei 200, de 25 de fevereiro de 1967. O Decreto 60.521, de 31 de maio daquele mesmo ano, reestruturou o Ministério da Aeronáutica e elevou o CTA a Comando Geral de Pesquisa e Desenvolvimento, criando simultaneamente o Instituto de Fomento e Coordenação Industrial para integrar o elo produtivo ao conjunto. O instituto nasceu, porém, de forma embrionária: sem sede própria, funcionou informalmente nas dependências do IPD enquanto aguardava sua ativação formal. O Decreto 64.200, de 14 de maio de 1969, institucionalizou essa fase transitória ao criar o Núcleo do Instituto de Fomento e Coordenação Industrial (NUIFI), diretamente subordinado ao Ministro da Aeronáutica.

A consolidação definitiva ocorreu em 1971. O Decreto 68.874, de 5 de julho, transformou o CTA em Centro Técnico Aeroespacial e confirmou o IFI como um de seus institutos constitutivos. Quinze dias depois, em 20 de agosto de 1971, o Ministro da Aeronáutica assinou a Portaria nº 065/GM2, ativando formalmente o NUIFI e determinando que, no prazo de noventa dias, fosse apresentado o anteprojeto de regulamento do instituto. Essa data marca, desde então, o aniversário institucional do IFI.

II.2. Área onde foi desenvolvido o programa de estágio

O estágio foi realizado no Laboratório de Medições Eletromagnéticas (DCTA/IFI/CMA-LE), pertencente ao Instituto de Fomento e Coordenação Industrial (IFI/DCTA), localizado na Praça Mal. Eduardo Gomes, 50, Vila das Acácias, São José dos Campos – SP.

II.3. O Estágio no Contexto da Empresa

O Laboratório de Medições Eletromagnéticas (DCTA/IFI/CMA-LE) conduz duas linhas principais de atividade. A primeira é a realização de ensaios de compatibilidade eletromagnética (EMC) segundo a norma MIL-STD-461 (Department of Defense, 2015), que estabelece os requisitos de interferência eletromagnética para equipamentos e subsistemas utilizados por plataformas militares. A segunda é a caracterização de antenas em câmara anecoica (Oliveira and Santos, 2020), cujos resultados permitem extrair grandezas como diagrama de irradiação e polarização.

O sistema desenvolvido durante o estágio insere-se na segunda linha de atividade e representa uma mudança qualitativa no processo de medição: a substituição do medidor de potência — que fornece apenas amplitude — por um VNA, capaz de medir amplitude e fase simultaneamente. Essa transição elimina a necessidade do método da antena linear girante (IEEE, 2021) para a determinação da razão axial, permite identificar o sentido da polarização elíptica — se à direita ou à esquerda, informação inacessível por medições exclusivamente de amplitude — e concentra, em uma única sessão de medição com duas

polarizações ortogonais, toda a informação necessária para a caracterização completa da antena sob teste. Ao longo do período de estágio, acompanhou-se também a equipe do laboratório na execução de ensaios de EMC, entre os quais os ensaios de susceptibilidade conduzida CS114 e CS116 e os ensaios de emissão irradiada (RE102) e susceptibilidade irradiada (RS103).

III. SISTEMA DE MEDIÇÃO AUTOMATIZADO

Neste capítulo, será descrito o sistema de medição de parâmetros S desenvolvido durante o estágio. O sistema controla um *Vector Network Analyzer* (VNA) e dois motores de posicionamento angular de antenas — transmissora (Tx) e receptora (Rx) — executando uma varredura de 0° a 360° para duas polarizações ortogonais da Tx. Em cada posição angular amostrada, os quatro parâmetros S (S11, S21, S12, S22) são adquiridos do VNA e armazenados. O sistema foi inteiramente implementado em Python (PyVISA Authors, 2024; Harris et al., 2020), com exceção do driver de baixo nível do motor Tx, que é um executável em C por depender da biblioteca proprietária NI FlexMotion. Na sequência, serão apresentados a arquitetura geral do sistema e cada um dos seus módulos, com os respectivos trechos de código.

III.1. Arquitetura Geral

O sistema é organizado em quatro módulos, cada um com responsabilidade bem delimitada. O ponto de entrada é `main.py`, que instancia o orquestrador e dispara o ciclo de medição. O orquestrador, implementado na classe `SYS` em `drivers/system.py`, coordena os demais componentes: o driver do VNA (`drivers/vna.py`), o driver do motor Tx (`drivers/tx.py`) e a instância de configuração (`config.py`). O motor Rx e seu encoder são controlados diretamente pelo orquestrador via VISA. A Tabela 1 resume os canais de comunicação com cada instrumento.

Instrumento	Protocolo	Endereço
VNA	VISA / SCPI sobre TCP/IP	<code>CFG.VNA_ADDRESS</code>
Encoder da Rx	VISA / GPIB sobre LAN	<code>CFG.RX_ENCODER_ADDRESS</code>
Fonte de alimentação da Rx	VISA / GPIB sobre LAN	<code>CFG.RX_POWERSUPPLY_ADDRESS</code>
Motor Tx	<code>subprocess</code> → <code>tx.exe</code> (NI FlexMotion)	—

Tabela 1: Instrumentos controlados pelo sistema e seus respectivos canais de comunicação.

Para executar o sistema, basta garantir que as dependências estejam instaladas e invocar o ponto de entrada:

```
1 from drivers.system import SYS
2
3 inst = SYS()
4 [filename_V, filename_H] = inst.measurement()
```

Listing 1: Ponto de entrada do sistema (`main.py`).

A chamada a `inst.measurement()` executa o ciclo completo e retorna os caminhos dos dois arquivos `.npz` gerados, um por polarização.

III.2. Configuração Centralizada

Todos os parâmetros do sistema — endereços VISA, potência de saída do VNA, *IF bandwidth*, arquivo de calibração e limites do *sweep* de frequência — estão reunidos em um único lugar: a instância global `CFG`, definida em `config.py`. A escolha de implementar essa estrutura como uma `NamedTuple` do Python é deliberada: tuplas nomeadas são imutáveis, de modo que nenhum módulo pode alterar `CFG` acidentalmente em tempo de execução. Para adaptar o sistema a um novo experimento ou a um hardware diferente, basta editar os valores no bloco de instanciação ao final do arquivo.

```
1 from typing import NamedTuple
2
3 class Config(NamedTuple):
4     """Parâmetros centralizados do sistema de medição (imutável)."""
```

```

5
6     # Endereços VISA
7     VNA_ADDRESS: str
8     RX_ENCODER_ADDRESS: str
9     RX_POWER_SUPPLY_ADDRESS: str
10
11     # Parâmetros do VNA
12     PWR: int      # Potência de saída (dBm)
13     IF: int       # IF bandwidth (Hz) - menor valor = menos ruído, sweep mais
14                  # lento
15     CAL: str      # Nome do arquivo de calibração no VNA
16
17     # Sweep de frequência
18     FSTART: float  # Hz
19     FSTOP: float   # Hz
20     POINTS: int
21
22 CFG = Config(
23     VNA_ADDRESS="TCPIP0::192.168.10.12::inst0::INSTR",
24     RX_ENCODER_ADDRESS="TCPIP0::192.168.10.10::gpib1,8::INSTR",
25     RX_POWER_SUPPLY_ADDRESS="TCPIP0::192.168.10.10::gpib1,6::INSTR",
26     PWR=-10,
27     IF=1000,
28     CAL="cal.cal",
29     FSTART=2e9,
30     FSTOP=10e9,
31     POINTS=101,
32 )

```

Listing 2: Definição da estrutura de configuração (config.py).

Note que o parâmetro IF controla diretamente o compromisso entre velocidade e ruído: valores menores produzem medições mais precisas, porém tornam cada *sweep* significativamente mais lento, o que é crítico considerando que o sistema realiza centenas de sweeps por varredura angular.

III.3. Driver do VNA

A comunicação com o VNA é encapsulada na classe VNA, em `drivers/vna.py`. A interface pública oferece três métodos: `setup`, que configura potência, *IF bandwidth* e define os quatro *traces* S; `configure_sweep`, que define a faixa de frequência e o número de pontos; e `measure_all_s`, que dispara um *sweep* e retorna os parâmetros S em formato complexo. Os métodos restantes são internos.

A inicialização da classe estabelece a sessão VISA e configura o modo de transferência de dados:

```

1 def __init__(self, resource: str):
2     rm = pyvisa.ResourceManager()
3     self.vna = rm.open_resource(resource)
4
5     # Sweeps com IF baixo ou muitos pontos podem levar vários segundos.
6     self.vna.timeout = 20000 # ms
7
8     # REAL,64: transferência binária em double precision (float64).
9     # SWAP: little-endian, compatível com numpy em x86/x64.
10    self.vna.write("FORM:DATA REAL,64")
11    self.vna.write("FORM:BORD SWAP")
12
13    # Reset para estado padrão - apaga configurações residuais.
14    self.vna.write("SYST:PRES")

```


Listing 3: Inicialização da sessão VISA e configuração do modo de transferência (`drivers/vna.py`).

A escolha de transferência binária em `REAL,64` merece atenção. O VNA é capaz de transmitir os dados tanto em formato ASCII quanto binário; entretanto, a representação ASCII de um número de ponto flutuante ocupa tipicamente entre 10 e 20 caracteres, ao passo que a representação binária de um `double` ocupa exatamente 8 bytes. Para um *sweep* com 101 pontos e quatro parâmetros S, cada um formado por pares (Re, Im), a diferença de volume de dados transmitidos é expressiva, e a leitura binária é, portanto, substancialmente mais rápida.

O disparo de cada *sweep* é realizado pelo método `_trigger`, que desabilita o modo contínuo do instrumento antes de iniciar uma única varredura e aguarda sua conclusão por meio do comando `*OPC?`:

```
1 def _trigger(self) -> None:
2     """Dispara um único sweep e aguarda conclusão (bloqueante)."""
3     self.vna.write("INIT:CONT OFF")
4     self.vna.write("INIT:IMM")
5     self.vna.query("*OPC?") # bloqueia até o sweep completar
6
7 def _get_complex_trace(self, name: str) -> np.ndarray:
8     """Lê um trace S e reconstrói o vetor complexo a partir dos pares (Re, Im)."""
9
10    self.vna.write(f"CALC1:PAR:SEL '{name}'")
11    raw = self.vna.query_binary_values(
12        "CALC1:DATA? SDATA", datatype="d", container=np.array
13    )
14    return raw[0::2] + 1j * raw[1::2]
15
16 def measure_all_s(self) -> dict:
17     """Dispara sweep e retorna S11, S21, S12, S22 e o vetor de frequência."""
18     self._trigger()
19     return {
20         "freq": self.freq,
21         "S11": self._get_complex_trace("S11"),
22         "S21": self._get_complex_trace("S21"),
23         "S12": self._get_complex_trace("S12"),
24         "S22": self._get_complex_trace("S22"),
25     }
```

Listing 4: Disparo de um único *sweep* e leitura de um *trace* S (`drivers/vna.py`).

O instrumento retorna os dados de cada *trace* como um vetor de números reais intercalados no formato `[Re0, Im0, Re1, Im1, ...]`. A reconstrução do vetor complexo é feita pelo fatiamento `raw[0::2] + 1j * raw[1::2]`, que seleciona alternadamente as partes real e imaginária. O vetor de frequências `self.freq` é calculado localmente via `np.linspace` no momento em que o *sweep* é configurado, pois o instrumento não altera os limites de frequência entre varreduras — lê-lo do instrumento a cada medição seria desnecessário e lento.

III.4. Controle do Motor Tx

O motor da antena Tx é um motor de passo controlado por uma placa NI FlexMotion. Por depender de uma biblioteca proprietária da National Instruments disponível apenas em Windows, o controle de baixo nível foi implementado em C e compilado como `tx.exe`. O driver Python, em `drivers/tx.py`, invoca esse executável via `subprocess` e verifica a convergência lendo um arquivo de cache.

A conversão entre graus e passos do motor é direta, dado que uma revolução completa corresponde a 6000 passos:

```

1 STEPS_PER_REV = 6000
2 DEG_PER_REV   = 360
3 MAX_ATTEMPTS  = 10
4 RETRY_DELAY   = 0.2 # s
5
6 def deg_to_steps(deg: float) -> int:
7     return int(round((STEPS_PER_REV / DEG_PER_REV) * deg))
8
9 def rotate_tx(target_deg: float) -> None:
10    """Move a antena Tx para a posição alvo (graus), com
11    retentativas até MAX_ATTEMPTS."""
12    target_steps = deg_to_steps(target_deg)
13
14    for attempt in range(1, MAX_ATTEMPTS + 1):
15        current_steps = _read_position() # lê arquivo de cache
16
17        if current_steps == target_steps:
18            print("[TX] Posição atingida.")
19            return
20
21        result = subprocess.run(
22            [EXECUTABLE, str(target_steps), CACHE_FILE],
23            cwd=CONTROL_DIR,
24        )
25        if result.returncode != 0:
26            raise RuntimeError("[TX] Erro ao executar tx.exe")
27
28        time.sleep(RETRY_DELAY)
29
30    raise RuntimeError(
31        f"[TX] Falha de convergência após {MAX_ATTEMPTS} tentativas."
32    )

```

Listing 5: Função principal de controle do motor Tx (`drivers/tx.py`).

O mecanismo de verificação de convergência merece ser explicado. O executável `tx.exe` recebe a posição alvo em passos e o caminho de um arquivo de texto; ao concluir o movimento, grava nesse arquivo a posição final efetivamente atingida pelo motor. Na chamada seguinte, `rotate_tx` lê esse arquivo e compara o valor com o alvo antes de invocar o executável novamente, evitando movimentos redundantes. Essa abordagem de comunicação por arquivo é necessária porque o executável não mantém estado entre execuções — cada chamada ao `subprocess` é um processo independente.

O executável, por sua vez, configura o perfil cinemático do movimento e aguarda sua conclusão em *polling* sobre os registradores de status da placa:

```

1 /* Configura e inicia movimento absoluto */
2 flex_load_acceleration(boardID, axis, NIMC_ACCELERATION, 4000, 0xFF);
3 flex_load_acceleration(boardID, axis, NIMC_DECELERATION, 4000, 0xFF);
4 flex_load_velocity(boardID, axis, 500, 0xFF);
5 flex_set_op_mode(boardID, axis, NIMC_ABSOLUTE_POSITION);
6 flex_load_target_pos(boardID, axis, targetPosition, 0xFF);
7 flex_start(boardID, axis, 0);
8
9 /* Aguarda conclusão do movimento */
10 do {
11     flex_read_pos_rtn(boardID, axis, &position);
12     flex_read_axis_status_rtn(boardID, axis, &axisStatus);
13     flex_read_csr_rtn(boardID, &csr);

```

```

14
15     if (csr & NIMC_MODAL_ERROR_MSG) {
16         flex_stop_motion(boardID, NIMC_VECTOR_SPACE1, NIMC_DECEL_STOP, 0);
17         /* ... tratamento de erro ... */
18     }
19 } while (!(axisStatus & (NIMC_MOVE_COMPLETE_BIT | NIMC_AXIS_OFF_BIT)));
20
21 /* Grava posição final no arquivo de cache */
22 FILE *f = fopen(outputPath, "w");
23 fprintf(f, "%d", position);
24 fclose(f);

```

Listing 6: Laço de espera pelo fim do movimento em `tx.c`.

O laço `do-while` termina quando o bit `NIMC_MOVE_COMPLETE_BIT` ou o bit `NIMC_AXIS_OFF_BIT` é ativado no registrador de status do eixo, indicando respectivamente que o movimento foi concluído normalmente ou que o eixo foi desabilitado por um erro. A placa utilizada é a de identificação (*board ID*) 3, eixo 1, com aceleração e desaceleração de 4000 passos/s² e velocidade máxima de 500 passos/s.

III.5. Orquestração das Medições

A classe `SYS`, em `drivers/system.py`, é o orquestrador de todo o experimento. Ao ser instanciada, ela inicializa o VNA — aplicando `preset`, configurando potência, *IF bandwidth* e *sweep* — e cria o diretório de saída para os arquivos de dados. O método público `measurement()` executa então dois ciclos de varredura angular completos, um para cada polarização da Tx, e retorna os caminhos dos arquivos gerados.

O núcleo do sistema é o método privado `_measure_full_rotation`, que gerencia a varredura angular da Rx. Ao ser chamado, ele posiciona a antena Tx na polarização solicitada, liga a fonte de alimentação do motor da Rx e entra em um laço de *polling* sobre o encoder angular:

```

1 pwr.write("VOLT 30")
2 pwr.write("CURR 1")
3 pwr.write("OUTP ON")
4
5 measure = 0
6 curr_angle = None
7 last_change_time = time()
8
9 while measure < 720:
10     angle = self._discretize_angle(
11         self._parse_angle(encoder.query("MEAS?"))
12     )
13
14     # Registra medição apenas quando o ângulo muda pelo menos 0,5°.
15     # A comparação em décimos evita erros de ponto flutuante.
16     if curr_angle is None or int(10 * angle) != int(10 * curr_angle):
17         angles.append(angle)
18         measure += 1
19         curr_angle = angle
20         last_change_time = time()
21
22     data = self.vna.measure_all_s()
23     s11_list.append(np.asarray(data["S11"]))
24     s21_list.append(np.asarray(data["S21"]))
25     # ...
26
27     # Watchdog: aborta se o ângulo estagnar por mais de 5 s.
28     if time() - last_change_time > self.WATCHDOG_TIMEOUT:

```

```

29         raise RuntimeError(
30             "Watchdog: ângulo estagnado - possível travamento."
31         )

```

Listing 7: Laço principal de varredura angular (`drivers/system.py`).

A condição de parada é `measure < 720`, valor escolhido para garantir cobertura angular completa mesmo com variações na velocidade de rotação do motor. A comparação `int(10 * angle) != int(10 * curr_angle)` é necessária para tratar corretamente a resolução de 0,5°: comparar diretamente valores de ponto flutuante como 0.5, 1.0, 1.5 pode produzir falsas diferenças devido a erros de representação. Ao multiplicar por 10 e truncar para inteiro, compara-se efetivamente os décimos de grau, o que é exato.

O *watchdog*, implementado pela verificação de `time() - last_change_time`, é uma salvaguarda contra travamentos mecânicos ou falhas de comunicação com o encoder. Caso o ângulo não evolua por mais de 5 segundos, uma `RuntimeError` é lançada e o laço é interrompido. Independentemente de como o laço termina — por conclusão normal, por `KeyboardInterrupt` ou por exceção —, a fonte de alimentação do motor é sempre desligada no bloco `finally`, garantindo que o motor não permaneça energizado sem supervisão.

III.5.1. Protocolo do Encoder Angular

O encoder da antena Rx responde ao comando GPIB MEAS? com uma cadeia de caracteres no formato `Xdddmmm`, em que `X` é um caractere líder ignorado, `ddd` representa os graus inteiros (com zeros à esquerda) e `mmm` os milésimos de grau. A interpretação é feita pelo método `_parse_angle`:

```

1 def _parse_angle(self, raw: str) -> float:
2     """Converte resposta bruta do encoder para graus decimais.
3
4     Formato: Xdddmmm, onde ddd = graus inteiros e mmm/1000 = fração.
5     Exemplo: 'X045500' -> 45 + 500/1000 = 45,5°
6     """
7     digits = raw.strip()[1:]
8     return round(int(digits[:3]) + int(digits[3:]) / 1000, 1)
9
10 def _discretize_angle(self, num: float) -> float:
11     """Arredonda para resolução de 0,5°.
12
13     Regra aplicada à parte fracionária (em décimos):
14         0-2 -> 0,0 (arredonda para baixo)
15         3-6 -> 0,5 (arredonda para metade)
16         7-9 -> 1,0 (arredonda para cima)
17     """
18     integer_part = int(num)
19     dec = int(10 * num) - 10 * integer_part
20
21     if dec <= 2:
22         frac = 0.0
23     elif dec <= 6:
24         frac = 0.5
25     else:
26         frac = 1.0
27
28     out = integer_part + frac
29     return 0.0 if int(out) == 360 else out

```

Listing 8: Interpretação da resposta bruta do encoder e discretização angular (`drivers/system.py`).

A discretização filtra o ruído natural do encoder, cujas leituras consecutivas na mesma posição física podem oscilar levemente. A normalização de 360° para 0° no final do método garante a consistência do

intervalo angular adotado, $[0^\circ, 360^\circ]$.

III.6. Formato dos Dados Gerados

Ao término de cada varredura angular, os dados coletados são empilhados em *arrays* NumPy e salvos em um arquivo `.npz` — o formato binário nativo do NumPy, sem compressão adicional. O nome do arquivo segue o padrão `YYYYMMDD_HHMMSS_⟨ID⟩_⟨POL⟩.npz`, em que ID é um identificador hexadecimal de 8 caracteres gerado por UUID4 e POL é V ou H. As duas medições de um mesmo experimento compartilham o mesmo *timestamp* e ID, facilitando sua associação.

Cada arquivo contém os seguintes *arrays*:

Array	Shape	Descrição
<code>angle</code>	$(N_a,)$	Ângulos amostrados (graus)
<code>freq</code>	$(N_f,)$	Frequências do <i>sweep</i> (Hz)
<code>S11, S21, S12, S22</code>	(N_a, N_f)	Parâmetros S (complexo)

Tabela 2: Conteúdo dos arquivos `.npz` gerados pelo sistema.

Cada linha das matrizes de parâmetros S corresponde a um ângulo e cada coluna a uma frequência, de modo que `S21[i, j]` é o parâmetro S_{21} medido no ângulo `angle[i]` e na frequência `freq[j]`. O carregamento dos dados em Python é imediato:

```
1 import numpy as np
2
3 d = np.load("../cache/sys/20250315_143022_A1B2C3D4_V.npz")
4 angulos = d["angle"] # shape (Na,)
5 s21      = d["S21"]   # shape (Na, Nf), complexo
```

Listing 9: Carregamento dos dados gerados.

IV. COMENTÁRIOS E CONCLUSÕES

O sistema desenvolvido durante o estágio cobre o ciclo completo de aquisição de parâmetros S em câmara anecoica: inicialização e configuração do VNA, controle dos motores de posicionamento das antenas Tx e Rx, sincronização entre posição angular e disparo do instrumento, e persistência dos dados em disco. O resultado é um conjunto de arquivos que registram os quatro parâmetros S em função do ângulo e da frequência, para as polarizações vertical e horizontal, prontos para análise posterior.

A etapa de processamento dos dados constitui um desenvolvimento natural a ser realizado futuramente. Por reunir amplitude e fase nas duas polarizações ortogonais, os dados adquiridos contêm toda a informação necessária para a caracterização completa da antena sob teste (IEEE, 2021): o diagrama de irradiação, a razão axial e o sentido da polarização elíptica — se à direita ou à esquerda — são deriváveis diretamente dos parâmetros S complexos coletados, sem necessidade de sessões adicionais de medição. A disponibilidade da fase é, em particular, o que torna possível a identificação do sentido da polarização, grandeza inacessível ao método da antena linear girante, que opera exclusivamente com amplitude. Pretende-se implementar essas rotinas futuramente. O formato de armazenamento adotado foi pensado precisamente para facilitar esse processamento: cada arquivo `.npz` reúne, em arrays NumPy indexados por ângulo e frequência, tudo o que é necessário para derivar essas grandezas sem qualquer etapa adicional de aquisição.

Referências

- Department of Defense. MIL-STD-461G: Requirements for the control of electromagnetic interference characteristics of subsystems and equipment. Military Standard MIL-STD-461G, 2015.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- IEEE. IEEE recommended practice for antenna measurements. IEEE Std 149-2021, Institute of Electrical and Electronics Engineers, 2021.
- Instituto de Fomento e Coordenação Industrial. Histórico do IFI. <https://ifi.dcta.mil.br/index.php/historico2>, 2025. Acesso em: maio de 2025.
- Sérgio Baptista de Oliveira and Carlos do Nascimento Santos. Implantação e automatização do laboratório de medições de compatibilidade eletromagnética para prestação de serviços aos Institutos de Pesquisa do Comando da Aeronáutica e empresas do setor aeroespacial. *Revista da UNIFA*, 33(1):65–77, 2020. doi: 10.22480/rev.unifa.v33n1.
- PyVISA Authors. PyVISA: Python package for support of the VISA standard. <https://pyvisa.readthedocs.io>, 2024.



INSTITUTO TECNOLÓGICO DE AERONÁUTICA

FOLHA DE FREQUÊNCIA DE ESTÁGIO CURRICULAR			Março		2026
IDENTIFICAÇÃO DO ESTAGIÁRIO					
NOME: Andryus Marzzona Fernandes Martins					
CURSO: Engenharia Eletrônica					
LOCAL DO ESTÁGIO: Instituto de Fomento e Coordenação Industrial (IFI-DCTA)					
FREQUÊNCIA					
DIA	PERÍODO	Nº DE HORAS	DIA	PERÍODO	Nº DE HORAS
01			16	Manhã	4
02	Manhã	4	17	Manhã	4
03	Manhã	4	18	Manhã	4
04	Manhã	4	19	Manhã	4
05	Manhã	4	20	Manhã	4
06	Manhã	4	21		
07			22		
08			23	Manhã	4
09	Manhã	4	24	Manhã	4
10	Manhã	4	25	Manhã	4
11	Manhã	4	26	Manhã	4
12	Manhã	4	27	Manhã	4
13	Manhã	4	28		
14			29		
15			30	Manhã	4
			31	Manhã	4
TOTAL DE HORAS: 88 (Março)					

FOLHA DE FREQUÊNCIA DE ESTÁGIO CURRICULAR				Abril	2026
IDENTIFICAÇÃO DO ESTAGIÁRIO					
NOME: Andryus Marzzona Fernandes Martins					
CURSO: Engenharia Eletrônica					
LOCAL DO ESTÁGIO: Instituto de Fomento e Coordenação Industrial (IFI-DCTA)					
FREQUÊNCIA					
DIA	PERÍODO	Nº DE HORAS	DIA	PERÍODO	Nº DE HORAS
01	Manhã Tarde	4 4	16	Manhã	4
02			17		
03			18		
04			19		
05			20		
06	Manhã Tarde	4 4	21		
07	Manhã Tarde	4 4	22	Manhã Tarde	4 4
08	Manhã Tarde	4 4	23	Manhã	4
09	Manhã	4	24	Manhã	4
10	Manhã	4	25		
11			26		
12			27		
13	Manhã Tarde	4 4	28		
14	Manhã Tarde	4 4	29		
15	Manhã Tarde	4 4	30		
			31		
TOTAL DE HORAS (Abril): 84					
TOTAL DE HORAS: 84 + 88 = 172					

Data: 14/05/2026

Aluno

**Orientador/Supervisor EMPRESA
com Carimbo**

Orientador/Supervisor ITA

Coordenador do Curso

Seção de Coordenação e Controle
Divisão de Assuntos Estudantis

Estagiário, entregar esta ficha devidamente preenchida e assinada na Divisão de Assuntos Estudantis.



AVALIAÇÃO DE ESTÁGIO

Empresa: Instituto de Fomento e Coordenação Industrial -- IFI/DCTA

Orientador/Supervisor do Estágio: Ten. Cássio Vinícius do Nascimento Santos

Nome do(a) Estagiário(a): Andryus Marzzona Fernandes Martins

Curso : Engenharia Eletrônica

Período de estágio: de 02/03/2026 a 24/04/2026

Resumo das atividades desenvolvidas: Desenvolveu-se um sistema de *software* em *Python* para automação de medições de parâmetros S em câmara anecoica, controlando um VNA e motores de posicionamento de antenas. Acompanhou-se também a equipe na realização de ensaios de EMC segundo a norma MIL-STD-461, em particular CS114, CS116, RE102 e RS103.

ITEM AVALIADO	FRACO	REGU-LAR	BOM	ÓTIMO	NÃO OBSER-VADO
Qualidade do trabalho: considerar a qualidade dos trabalhos executados pelo estagiário, tendo em vista as condições oferecidas e as tarefas executadas em nível universitário de Engenharia.				X	
Interesse e Aprendizado Prático: considerar o interesse e dedicação do estagiário pelo melhor aproveitamento do estágio inclusive observando a capacidade e o esforço de aprendizagem demonstrada.				X	
Planejamento e Organização: capacidade de planejar e organizar as ações relativas às atividades, tarefas e os objetivos do estágio na empresa ou instituição; considerar o rendimento e o atendimento de prazos.				X	
Engenhosidade/Criatividade: capacidade de sugerir, projetar ou executar modificações ou inovações, no contexto do estágio, que beneficiem a Empresa.				X	
Iniciativa e Independência: procurar por novas soluções sem prévia orientação.				X	
Conhecimentos técnicos: conhecimentos apresentados e que tenham se mostrado adequados para boa execução das tarefas desenvolvidas.				X	
Cooperação e trabalho em equipe: capacidade de manter-se disponível para contribuir com sua ajuda quando solicitada e em sua integração em trabalho de equipe.				X	
Responsabilidade: cumprimento das atividades dispensando a constante coordenação dos chefes.				X	
Sociabilidade: capacidade de bem se relacionar com as pessoas.				X	
Liderança: capacidade de direção e coordenação, fazendo-se seguir ou atender com naturalidade.				X	
Pontualidade: atendimento dos horários a que está sujeito.				X	

Obs. Acrescentar algum outro item de avaliação que julgar importante considerar.

Data: 15/05/2026

Aluno

Orientador/Supervisor EMPRESA

Orientador/Supervisor do ITA

Coordenador do Curso/ITA